
ASSESSMENT REPORT

**Develop Incident Response Plan,
Analyze Security Monitoring Dashboard,
Recommend Mitigation Strategies
and Technical Justification**

DevSecOps Engineering and Application Security

Name	T G JAI KOUSICK
Register Number	192421367
Subject	DevSecOps Engineering and Application Security
Subject Code	CSA5802
Department	B.Tech Information Technology

1. Develop Incident Response Plan

An Incident Response Plan (IRP) defines the structured approach an organisation follows to detect, contain, eradicate, recover from, and learn from security incidents. In a DevSecOps and cloud-native environment the plan must support rapid automation, clear ownership, and continuous improvement while aligning with frameworks such as NIST SP 800-61 and ISO/IEC 27035.

1.1 IRP Objectives

- Minimise impact on confidentiality, integrity, and availability of systems and data.
- Enable fast, consistent, and auditable response actions.
- Preserve forensic evidence for investigation and potential legal proceedings.
- Restore normal operations within defined Recovery Time Objectives (RTO) and Recovery Point Objectives (RPO).
- Feed lessons learned back into prevention, detection, and pipeline security controls.

1.2 Incident Response Lifecycle (NIST-aligned)

Phase	Key Activities	Primary Owners
1. Preparation	Policies, playbooks, tooling, training, communication trees, asset inventory, runbooks, tabletop exercises	CISO / Security Ops, Platform, Legal
2. Detection & Analysis	Alert triage, severity classification, scope assessment, initial indicators of compromise (IoCs), evidence collection	SOC / Security On-call, SRE
3. Containment	Short-term isolation (network, identity, workload), long-term containment, prevent lateral movement	Security Ops, Platform / Cloud Ops
4. Eradication	Remove malware/backdoors, revoke compromised credentials, patch vulnerabilities, rebuild from known-good images	Security Ops, Engineering
5. Recovery	Restore services from clean backups / GitOps desired state, progressive re-enablement, enhanced monitoring	SRE / Platform, Application teams
6. Post-Incident Activity	Root-cause analysis, timeline, lessons learned, update playbooks & controls, metrics (MTTD/MTTR)	All stakeholders + Leadership

Table 1: Incident Response Lifecycle Phases

1.3 Severity Classification (Example)

Severity	Criteria (examples)	Response Target
P1 – Critical	Active data breach, ransomware, full service outage, confirmed privileged account compromise	Immediate (15–30 min)
P2 – High	Suspected breach, significant service degradation, lateral movement indicators	1 hour
P3 – Medium	Isolated malware, policy violations, non-critical vulnerability exploitation attempts	4–8 hours
P4 – Low	Suspicious but contained activity, false-positive prone alerts needing review	Next business day

Table 2: Incident Severity Levels and Response Targets

1.4 Core Playbooks (DevSecOps / Cloud-Native Focus)

- **Compromised credentials / identity:** Force logout, revoke tokens/keys, rotate secrets via Vault/KMS, review CloudTrail/Audit logs, enable MFA enforcement.
- **Malicious or vulnerable container:** Quarantine namespace/workload via NetworkPolicy, drain nodes if needed, rebuild from signed GitOps image, re-scan registry.
- **Ransomware / data encryption event:** Isolate affected systems, preserve volatile evidence, activate immutable backups, communicate per legal/comms plan.
- **Supply-chain / poisoned artifact:** Block image digests at admission controller, revoke signing keys if compromised, force rebuild from clean source, notify downstream consumers.
- **DDoS / availability attack:** Engage cloud provider DDoS protection / WAF, scale or shift traffic, rate-limit, preserve logs for analysis.
- **Insider / anomalous privileged activity:** Suspend account, review just-in-time access logs, preserve audit trail, escalate to HR/Legal as required.

1.5 Communication & Roles

Define an Incident Commander, Technical Lead, Communications Lead, and Scribe. Maintain an up-to-date on-call roster and escalation matrix (Security → Platform → Engineering → Legal/PR → Executive). Use a dedicated incident channel (e.g., Slack/Teams + bridge) and a single source of truth (ticket + status page). External disclosure follows legal and regulatory requirements (e.g., data-breach notification timelines).

2. Analyze Security Monitoring Dashboard

A security monitoring dashboard aggregates signals from SIEM, observability platforms, CSPM, runtime security, and pipeline tools into a single operational view. Effective analysis converts raw alerts into prioritised, actionable incidents and continuous risk insights.

2.1 Essential Dashboard Panels / Metrics

Panel Category	Example Metrics / Visualisations	Purpose
Alert Volume & Health	Open alerts by severity, MTTA/MTTD trends, false-positive rate, alert age	SOC workload & detection quality
Identity & Access	Failed logins, privilege escalations, anomalous geo/IP, JWT/token anomalies	Detect credential abuse
Runtime Threats	Falco/Sysdig rule hits, unexpected process/network activity, container escapes	Live workload protection
Vulnerability Posture	Critical/High CVEs by asset, EPSS-weighted risk, KEV matches, aging findings	Prioritise remediation
Pipeline / Supply Chain	Failed security gates, unsigned images, secret leaks blocked, SBOM drift	Shift-left effectiveness
Network & Edge	WAF blocks, DDoS indicators, unusual egress, east-west anomalies	Perimeter & lateral movement
Compliance & Drift	Policy violations (OPA/Kyverno), CIS benchmark failures, config drift	Continuous compliance
Business Impact	Affected services, customer impact estimate, SLA/RTO burn-down	Prioritise response effort

Table 3: Security Monitoring Dashboard – Key Panels

2.2 Analysis Methodology

- **Triage by severity and asset criticality** — P1/P2 alerts on crown-jewel systems first.
- **Correlate across sources** — Link identity, network, runtime, and pipeline events into a single timeline (SIEM correlation rules / SOAR).
- **Enrich with context** — Asset ownership, data classification, recent deployments (GitOps history), known IoCs/threat intel.
- **Determine true vs false positive** — Validate against baselines; tune noisy rules to reduce alert fatigue.
- **Scope and blast radius** — Identify affected accounts, workloads, namespaces, and data stores.
- **Escalate or auto-remediate** — Trigger playbooks (manual or SOAR) and update the incident ticket with evidence links.
- **Trend and posture review** — Weekly/monthly review of MTTD, MTTR, open Critical vulnerabilities, and recurring attack patterns.

2.3 Example Dashboard Interpretation Scenarios

- Spike in failed logins + new geo + successful privilege escalation → likely credential compromise; invoke identity playbook.
- Sudden increase in Critical CVEs on production images after a dependency update → block further deploys, force rebuild with patched versions.
- Falco alerts for unexpected shell in container + unusual egress → possible container breakout or crypto-mining; isolate workload.
- Rising unsigned image admission rejections → signing pipeline regression; investigate CI Cosign/ key configuration.
- Sustained high false-positive rate on a specific rule → schedule rule tuning and suppress with documented justification.

3. Recommend Mitigation Strategies

Mitigation strategies reduce the likelihood and impact of incidents. They span preventive controls, detective controls, and response/recovery capabilities, and should be prioritised by risk (likelihood × impact).

3.1 Preventive Mitigations

- Enforce least-privilege IAM and short-lived credentials (OIDC federation, workload identity); eliminate long-lived static keys.
- Mandate signed commits, protected branches, required security status checks, and CODEOWNERS on application and GitOps repos.
- Admit only signed, scanned images via admission controllers (Cosign verification + vulnerability policy).
- Apply network policies, service mesh mTLS, and zero-trust segmentation between namespaces and services.
- Keep dependencies and base images current via automated SCA and image rebuild pipelines; maintain SBOMs.
- Encrypt data at rest and in transit; manage secrets exclusively through Vault / cloud KMS + External Secrets Operator.
- Harden Kubernetes (Pod Security Standards, CIS benchmarks) and cloud accounts (CSPM continuous assessment).

3.2 Detective Mitigations

-
- Centralised logging and SIEM with correlation rules covering identity, API, runtime, and pipeline events.
 - Runtime threat detection (Falco or equivalent) with tuned rules for container and host behaviour.
 - Continuous vulnerability re-scanning of running workloads and registries; alert on new KEV / high-EPSS findings.
 - Anomaly detection on authentication, network flows, and resource usage (CPU/memory spikes indicative of mining or exfiltration).
 - Integrity monitoring of critical configs and GitOps drift detection.

3.3 Responsive & Recovery Mitigations

- Automated containment playbooks (SOAR or scripted) for common high-confidence alerts (e.g., isolate pod, revoke token).
- Immutable, tested backups and GitOps-driven rebuild capability to meet RTO/RPO targets.
- Progressive delivery with automated rollback on security or health regressions.
- Regular tabletop and technical exercises of the IRP; measure and improve MTTD/MTTR.
- Clear exception and risk-acceptance process with expiry dates and compensating controls.

3.4 Prioritisation Matrix (Example)

Prioritise mitigations that address high-likelihood / high-impact risks first (e.g., credential hygiene, image signing + admission control, secret management, and runtime detection). Medium-risk items follow in subsequent sprints; low-risk items are scheduled as capacity allows. Track residual risk after each mitigation.

4. Technical Justification

The recommended plan, dashboard design, and mitigations are grounded in established security engineering principles and observed effectiveness in cloud-native environments.

4.1 Why a Structured IRP Works

Unstructured response leads to inconsistent actions, evidence loss, and prolonged dwell time. A documented lifecycle with severity levels, playbooks, and clear roles reduces Mean Time to Respond and ensures regulatory and contractual obligations are met. Alignment with NIST SP 800-61 provides a widely accepted baseline that auditors and partners recognise. Automation of repetitive containment steps further compresses response time while preserving human oversight for complex decisions.

4.2 Why Dashboard-Driven Monitoring Is Essential

Modern environments generate high volumes of telemetry. Without aggregation, correlation, and prioritisation, analysts suffer alert fatigue and miss true positives. Dashboards that combine severity, asset criticality, and cross-source context enable risk-based triage. Metrics such as MTTD, MTTR, and escape rate provide objective feedback for continuous improvement of both detection rules and the IRP itself.

4.3 Justification of Key Mitigations

- **Signed artifacts + admission control:** Prevents deployment of tampered or unscanned images; enforces supply-chain integrity (SLSA-aligned).
- **Least-privilege + short-lived credentials:** Limits blast radius of any single compromise; removes standing privileges that attackers seek.
- **Network segmentation & mTLS:** Constrains lateral movement even if a workload is compromised.
- **Runtime detection:** Catches threats that bypass static and pre-deploy controls (zero-days, living-off-the-land techniques).
- **GitOps + immutable infrastructure:** Enables reliable, auditable recovery by reconciling to a known-good desired state rather than manual fixes.
- **Automated SCA and continuous re-scan:** Addresses the reality that new vulnerabilities are disclosed daily; reduces window of exposure.
- **Tabletop exercises and measured KPIs:** Convert the IRP from a static document into a living capability that improves with each exercise and real incident.

4.4 Alignment with DevSecOps Principles

Security is embedded as code (policies, playbooks, detection rules) and executed continuously. Feedback from incidents and dashboard metrics flows back into pipeline gates and runtime controls, closing the loop between prevention, detection, and response. This approach improves both security posture and delivery confidence without relying solely on manual reviews.

5. Conclusion

A complete security operations capability in a DevSecOps setting requires: (1) a clear, exercised Incident Response Plan with severity-based playbooks; (2) a monitoring dashboard that enables rapid, context-rich triage and continuous posture visibility; (3) layered mitigation strategies that reduce likelihood and impact; and (4) technical justification rooted in risk reduction, automation, and measurable outcomes (MTTD, MTTR, residual risk). Together these elements form a resilient, auditable, and continuously improving security system.

Key Deliverables Summary

- Incident Response Plan covering Preparation through Post-Incident Activity, with severity model and cloud-native playbooks.
 - Security Monitoring Dashboard design with essential panels, analysis methodology, and interpretation scenarios.
 - Prioritised preventive, detective, and responsive mitigation strategies.
 - Technical justification linking each recommendation to risk reduction and DevSecOps principles.
-